Week 3, Day 3: Resilience4j and Binary Trees

Prepared for Upendra Kumar Sharma | 3 to 4 hours | Main topic: circuit breaker, retry, bulkhead, rate limiter, time limiter, fallbacks | DSA: Binary tree basics

Today's Schedule

- **0:00 to 0:40** Study why resilience patterns exist and the circuit breaker in depth (sections 1 and 2).
- **0:40 to 1:30** Study retry, bulkhead, rate limiter and time limiter (section 3).
- **1:30 to 2:00** Study fallbacks, how the patterns combine, and observability (sections 4 to 6).
- **2:00 to 3:10** DSA: solve the 4 binary tree problems (section 8). Spend at most 20 minutes per problem before reading the hint.
- **3:10 to 3:40** Answer the interview questions aloud (section 9) and add 10 flashcards.
- **Optional (60 to 90 min):** do the hands-on exercise in section 7 using yesterday's slow inventory service.

1. Why Resilience Patterns

- In a distributed system a dependency **will** be slow or down at some point. The goal is not to prevent failure but to **contain** it, so one sick service does not take down its callers.
- Yesterday's cascading failure: a slow inventory service holds order-service threads, which exhausts its thread pool, and then everything calling order-service stalls.
- **Resilience4j** is the standard lightweight library for this in Spring Boot. It replaced **Netflix Hystrix**, which is no longer maintained.
- **Modules:** CircuitBreaker, Retry, Bulkhead (semaphore and thread-pool), RateLimiter, TimeLimiter and Cache.
- **Setup in Spring Boot 3:** add resilience4j-spring-boot3, spring-boot-starter-aop (the annotations work through proxies, so the self-invocation pitfall from Week 2 applies) and spring-boot-starter-actuator.
- **Spring Cloud CircuitBreaker** is a thin abstraction (CircuitBreakerFactory) over Resilience4j. Spring Cloud Gateway also has a CircuitBreaker filter with a fallback URI.

2. The Circuit Breaker

The idea: like an electrical breaker, it **stops calling a failing dependency** for a while, so callers fail fast instead of waiting, and the dependency gets time to recover.

Three main states

- **CLOSED:** normal. Calls go through and the breaker records successes and failures in a sliding window.
- **OPEN:** the failure rate (or slow-call rate) passed the threshold. Calls are **rejected immediately** with CallNotPermittedException, without touching the dependency, for a set wait time.
- **HALF_OPEN:** after the wait time, a **limited number of trial calls** are allowed. If their failure rate is below the threshold the breaker goes back to CLOSED, otherwise it returns to OPEN.
- There are also DISABLED and FORCED_OPEN states for operations and testing.

```
CLOSED --(failure or slow rate >= threshold)--> OPEN
OPEN --(wait duration elapsed)--> HALF_OPEN
HALF_OPEN --(trial calls succeed)--> CLOSED
HALF_OPEN --(trial calls fail)--> OPEN
```

Key settings (defaults in brackets)

- **slidingWindowType:** COUNT_BASED (last N calls) or TIME_BASED (last N seconds).
- **slidingWindowSize** [100] and **minimumNumberOfCalls** [100]: the breaker will not trip until it has seen enough calls, so set a lower minimum for low-traffic services.
- **failureRateThreshold** [50 percent].
- **slowCallDurationThreshold** [60 s] and **slowCallRateThreshold** [100 percent]: treat very slow calls as failures. This is how a slow dependency opens the breaker even when it never returns errors.
- **waitDurationInOpenState** [60 s].
- **permittedNumberOfCallsInHalfOpenState** [10].
- **recordExceptions and ignoreExceptions:** decide which exceptions count as failures. **Do not count business errors** such as 404 Not Found or validation failures, otherwise normal user mistakes can open the breaker.

```
resilience4j:
  circuitbreaker:
    instances:
      inventory:
        slidingWindowType: COUNT_BASED
        slidingWindowSize: 20
        minimumNumberOfCalls: 10
        failureRateThreshold: 50
        slowCallDurationThreshold: 2s
        slowCallRateThreshold: 80
        waitDurationInOpenState: 30s
        permittedNumberOfCallsInHalfOpenState: 3
        automaticTransitionFromOpenToHalfOpenEnabled: true
        ignoreExceptions:
          - com.example.order.StockNotFoundException
```

Where to put it: around each call to a **remote dependency**, with **one breaker instance per dependency**, so a failing payment service does not trip the breaker for inventory.

3. Retry, Bulkhead, Rate Limiter and Time Limiter

Retry

- Repeats a failed call. Settings: **maxAttempts** [3, including the first call], **waitDuration** [500 ms], **enableExponentialBackoff** with a multiplier, **enableRandomizedWait** (jitter), and which exceptions to retry or ignore.
- Retry only **transient** failures and only **idempotent** operations (or ones protected by an idempotency key). Never retry on 4xx client errors.
- Watch out for **retry amplification** across layers, and make sure the total of all attempts fits inside the caller's timeout budget.

```
resilience4j:
  retry:
    instances:
      inventory:
        maxAttempts: 3
        waitDuration: 200ms
        enableExponentialBackoff: true
        exponentialBackoffMultiplier: 2
        enableRandomizedWait: true
        retryExceptions:
          - java.io.IOException
          - java.util.concurrent.TimeoutException
```

Bulkhead

- Named after ship compartments: **limit the resources one dependency can use**, so a slow one cannot consume all threads.
- **Semaphore bulkhead:** limits the number of **concurrent calls** (maxConcurrentCalls [25], maxWaitDuration [0]). It runs on the caller's thread, is cheap, and is the usual choice for blocking code. When full it throws BulkheadFullException.
- **Thread-pool bulkhead:** runs calls in a **separate bounded thread pool** with a queue, and returns a CompletableFuture. Stronger isolation, more overhead.
- Size the limits from real traffic: concurrency is roughly requests per second multiplied by the average latency in seconds.

```
resilience4j:
  bulkhead:
    instances:
      inventory:
        maxConcurrentCalls: 20
        maxWaitDuration: 0
```

Rate Limiter

- Limits **how many calls are allowed per time period**: **limitForPeriod**, **limitRefreshPeriod** and **timeoutDuration** (how long a caller waits for a permit). When exceeded it throws RequestNotPermitted.
- Use it to **protect a service from overload**, or to **stay within a third-party API quota**.
- It works **per application instance**. For a limit across all instances you need a shared store such as Redis, for example the RequestRateLimiter filter in Spring Cloud Gateway. Respond to clients with **429 Too Many Requests** and a Retry-After header.

Time Limiter

- Puts a **maximum duration** on an asynchronous call (**timeoutDuration** [1 s], **cancelRunningFuture**). It works with **CompletableFuture** or Future, so the annotated method must return one.
- It does not replace HTTP client timeouts. The underlying socket call may keep running, so **always set connect and read timeouts on the client as well**.

4. Fallbacks

- A **fallback method** runs when the call fails, is rejected by the breaker, or the bulkhead is full. It must have the **same return type** and the **same parameters plus an exception parameter**. You can define several, and the most specific exception type wins.
- **Good fallbacks:** cached or last-known data, a safe default, a reduced feature, or queuing the work for later.
- **Bad fallbacks:** one that calls another unreliable service (it fails too), or one that **pretends a write succeeded**. For example, never return a successful payment because the payment service was down. Be honest with the caller.
- Choosing a fallback is a **business decision**. Ask what the user should see.
- Remember that **an open breaker plus no fallback** still produces an error, which is correct. Fast failure is the main benefit.

```
@Service
public class StockService {
    private final InventoryClient inventoryClient;
    public StockService(InventoryClient inventoryClient) {
this.inventoryClient = inventoryClient; }

    @CircuitBreaker(name = "inventory", fallbackMethod = "stockFallback")
    @Retry(name = "inventory")
    @Bulkhead(name = "inventory")
    public StockResponse getStock(String sku) {
        return inventoryClient.getStock(sku);
    }

    // same return type, same parameters, plus the exception
    private StockResponse stockFallback(String sku, Throwable ex) {
        return StockResponse.unknown(sku);     // caller shows availability
unknown
    }
}
```

5. How the Patterns Combine

- **Default order of the Resilience4j annotations** (outermost to innermost): Retry, then CircuitBreaker, then RateLimiter, then TimeLimiter, then Bulkhead, then your method. Each retry attempt therefore passes through the circuit breaker, and an open breaker stops further retries quickly. You can change the order through aspect order properties.
- **A sensible recipe for a remote call:**

1. A **timeout** on the HTTP client (always).
2. A **bulkhead** so one dependency cannot take all threads.
3. A **circuit breaker** so a failing dependency is skipped.
4. A **limited retry** with backoff and jitter, for transient errors on idempotent calls.
5. A **fallback** that degrades gracefully.

- **Beyond Resilience4j:** caching responses, load shedding (reject low-priority work when overloaded), graceful degradation, health checks that remove bad instances, and **asynchronous messaging** so some work does not depend on the downstream being up.
- **Spring Retry** (@Retryable) is an alternative for plain retries, but Resilience4j gives you one consistent set of tools and metrics.

6. Observability and Testing

- **Actuator endpoints:** /actuator/circuitbreakers, /actuator/circuitbreakerevents, /actuator/retries and /actuator/bulkheads show the current state and recent events. Register the breaker health indicator to include it in /actuator/health.
- **Metrics:** Micrometer exports counters and gauges such as resilience4j_circuitbreaker_state, call counts by outcome, and not-permitted calls. Graph them in Grafana and **alert when a breaker is OPEN for longer than expected**.
- **Log state changes** so you can match an incident timeline to breaker transitions.
- **Test failures on purpose:** use WireMock or a test double to return errors and delays, and assert that the breaker opens, the fallback is used and the breaker closes again. Chaos testing in a staging environment is the next step.
- **Tune with data:** thresholds that look reasonable on paper often need adjusting after you watch real traffic.

7. Optional Hands-On (60 to 90 Minutes)

Using order-service, the Feign client and the slow inventory endpoint from Day 2:

1. Add resilience4j-spring-boot3, spring-boot-starter-aop and actuator. Expose the circuit breaker endpoints.
2. Annotate the stock lookup with CircuitBreaker, Retry and Bulkhead and use the YAML in this guide.
3. Make inventory return 500 errors, or sleep longer than the timeout. Send 30 requests (a shell loop is enough) and watch /actuator/circuitbreakers move from CLOSED to OPEN.

4. While it is OPEN, confirm that calls return instantly via the fallback and that inventory receives no traffic.
5. Fix inventory and wait for the open duration. Watch HALF_OPEN and then CLOSED.
6. Run 50 parallel requests with and without the bulkhead and compare how many order-service threads are blocked.
7. Write down what you saw with numbers (response times, state changes). It is a ready-made interview story.

8. DSA: Binary Trees (4 Problems)

All problems use this node class.

```
public class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}
```

Traversals to know cold

- **Preorder:** node, left, right. **Inorder:** left, node, right (sorted order for a binary search tree). **Postorder:** left, right, node. **Level order:** breadth-first with a queue.
- Recursive versions use O(h) stack space, where h is the height, which is O(n) for a skewed tree. Know how to write the iterative inorder with an explicit stack too.

Problem 1: Maximum Depth of Binary Tree

- **Idea:** the depth of a tree is 1 plus the larger depth of its two subtrees. An empty tree has depth 0. A BFS that counts levels also works.
- **Complexity:** O(n) time, O(h) space.
- **Edge cases:** empty tree, single node, completely skewed tree.

```
public int maxDepth(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

Problem 2: Invert Binary Tree

- **Idea:** invert both subtrees, then swap them. Same shape of recursion as depth.
- **Complexity:** O(n) time, O(h) space.
- **Edge cases:** empty tree, single node, a node with only one child.

```
public TreeNode invertTree(TreeNode root) {
    if (root == null) return null;
    TreeNode left = invertTree(root.left);
    TreeNode right = invertTree(root.right);
    root.left = right;
    root.right = left;
    return root;
}
```

Problem 3: Same Tree

- **Idea:** two trees are the same if both are empty, or both roots have equal values and their left subtrees match and their right subtrees match. Check the null cases first.
- **Complexity:** O(n) time, O(h) space.
- **Edge cases:** one empty and one not, equal values but different shapes.
- **Follow-up:** Subtree of Another Tree and Symmetric Tree use the same check.

```
public boolean isSameTree(TreeNode p, TreeNode q) {
    if (p == null && q == null) return true;
    if (p == null || q == null || p.val != q.val) return false;
    return isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
}
```

Problem 4: Binary Tree Level Order Traversal

- **Idea:** BFS with a queue. At the start of each round, record the **queue size**, which is exactly the number of nodes on the current level, and process that many nodes while adding their children.
- **Complexity:** O(n) time, O(w) space, where w is the maximum width of the tree.
- **Edge cases:** empty tree (return an empty list), single node.
- **Variations:** zigzag level order, right-side view, and average of each level, all by adjusting how you read each level.

```
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;
    Queue<TreeNode> queue = new ArrayDeque<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
        int size = queue.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        result.add(level);
    }
    return result;
}
```

9. Interview Questions to Answer Aloud

1. What problem does a circuit breaker solve? Describe its three states and transitions.
2. Which settings control when a breaker opens? Why does minimumNumberOfCalls matter?
3. Why should slow calls count as failures, and how is that configured?
4. Which exceptions should not count toward the failure rate, and why?
5. When is retry safe, and what are exponential backoff, jitter and retry amplification?
6. Semaphore bulkhead vs thread-pool bulkhead: what is the difference, and when would you use each?
7. How does a rate limiter differ from a bulkhead? What is the problem with per-instance rate limiting?
8. What does a time limiter do, and why do you still need HTTP client timeouts?
9. What makes a good fallback and a bad one? Give an example of each.
10. In which order do the Resilience4j annotations wrap a call, and why does it matter?
11. Why does the self-invocation pitfall also affect @CircuitBreaker and @Retry?
12. How do you monitor circuit breakers in production, and what would you alert on?
13. A downstream service is slow but not returning errors. Walk through everything that protects your service.
14. How does Resilience4j relate to Hystrix, and to Spring Cloud CircuitBreaker?

10. End-of-Day Checklist

- [ ] I can draw the circuit breaker states and explain the key settings.
- [ ] I can explain retry, bulkhead, rate limiter and time limiter, and when each is appropriate.
- [ ] I can write a fallback method and say what makes a fallback honest.
- [ ] I can state the default decoration order and the recipe for a remote call.
- [ ] I know which actuator endpoints and metrics show the breaker state.
- [ ] I solved all 4 tree problems and can explain the four traversals.
- [ ] (Optional) I tripped a breaker on purpose and recorded the numbers.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Week 3, Day 4): distributed transactions (saga with choreography and orchestration), the outbox pattern, CQRS and idempotency, plus binary search tree problems.